

Erwan Franck KAAWAR

Ivan LAI

Tommy LIM

Anthime L'HERMINE

Maxime KOBRIN



Groupe AFR3-F

GreenTasks – Rapport de projet

TI616 — Numérique Durable

Sommaire

Table des matières

Phase 1 - Présentation du projet	3
1.1 - Identifier le besoin et la proposition de valeur	3
1.2 - Définir le MVP.....	3
1.3 - Définir les utilisateurs et leurs rôles	4
1.4 - Identifier les données manipulées	4
1.5 - Rédiger des user stories	5
1.6 - Construire un mini backlog	5
Phase 2 - Architecture fonctionnelle et conception	6
2.1 - Diagramme de cas d'utilisation	6
2.2 - Diagramme de classes	6
2.3 - Diagramme de séquence - Connexion utilisateur	7
Phase 3 – Choix des technologies justifiés	8
Phase 4 – Implémentation	9
4.1 - Développement front-end	9
4.2 - Développement back-end et accès à la base de données	10
4.3 - Déploiement du site web	11
Phase 5 – Analyse de l'empreinte carbone	11
5.1 - Outils de mesure	11
5.2 - Protocole d'analyse.....	11
Phase 6 — Tests et validation	16
6.1 - Tests fonctionnels	16
6.2 - Tests de performance.....	16
6.3 - Tests de sécurité minimaux	17

Phase 1 - Présentation du projet

1.1 - Identifier le besoin et la proposition de valeur

- 1) L'accompagnement des élèves dans leurs projets (avancement, etc ...)
- 2) Un site web qui permet la gestion des tâches dans les projets.
- 3) Il s'adresse d'un côté aux enseignants / professeurs, et de l'autre aux étudiants.
- 4) Le fait d'avoir un objectif commun et visible pour tous, pour ne pas perdre de vue les objectifs à réaliser ainsi qu'un retour de la part des enseignants.
- 5) Notre projet est réaliste pour une équipe de 5 étudiants car il repose sur des fonctionnalités CRUD (Créer, Lire, Mettre à jour, Supprimer) simples.

1.2 - Définir le MVP

- 1) Fonction centrale : Gestion de tâches au sein de groupes d'étudiants supervisés par un professeur.
- 2) Fonctionnalités indispensables :
 - Création de compte et connexion.
 - Création et gestion de groupes par le professeur
 - Assignation et retrait d'élèves dans les groupes par le professeur
 - Ajout, modification et affichage (CRUD) des tâches par les élèves.
 - Consultation de l'ensemble des tâches d'un groupe par le professeur.
 - Feedback par le prof (validé/partiel/refusé)
 - Gestion des utilisateurs par l'administrateur
- 3) Fonctionnalités reportées :
 - Système de notifications ou rappels par email (trop énergivore).
 - Modification et suppression de son propre compte (étudiant ou professeur)

1.3 - Définir les utilisateurs et leurs rôles

Action	Visiteur	Eleve	Enseignant	Administrateur
Consulter les pages publiques	Oui	Oui	Oui	Oui
Créer un compte	Oui	Oui	Oui	Non
Se connecter	Oui	Oui	Oui	Oui
Modifier son profil	Non	Non	Non	Non
Supprimer un compte	Non	Non	Non	Oui
Consulter les données métier	Non	Oui	Oui	Non
Créer une donnée métier	Non	Oui	Oui	Non
Modifier une donnée métier	Non	Oui	Oui	Non
Supprimer une donnée métier	Non	Oui	Oui	Non
Gérer les utilisateurs	Non	Non	Non	Oui

1.4 - Identifier les données manipulées

- 1) Utilisateurs : Nom, email, mot de passe, rôle (élève/prof/admin).
- 2) Entité Métier (Tâche) : Titre, description, statut (à faire/en cours/fini), échéance, lien vers l'élève créateur et le groupe.
- 3) Relation entre les données :
 - Professeur ↔ Groupes : Un enseignant peut créer et posséder plusieurs groupes. Un groupe appartient à un seul professeur.
 - Groupe ↔ Élèves : Un groupe contient plusieurs élèves. Un élève est rattaché à un groupe.
 - Élève ↔ Tâches : Un élève crée et gère plusieurs tâches au sein de son groupe.
 - Professeur ↔ Tâches : Le professeur a une relation de lecture seule sur toutes les tâches liées aux groupes qu'il a créés, lui permettant de superviser l'avancement global.
- 4) Sobriété des données : Limitation des champs au strict nécessaire (pas d'avatars images lourds, pas de pièces jointes volumineuses) pour réduire le stockage

1.5 - Rédiger des user stories

US1 : En tant que visiteur, je veux créer un compte afin d'accéder au site

US2 : En tant qu'utilisateur, je veux me connecter afin d'accéder à mes fonctionnalités

US3 : En tant qu'élève, je veux créer une tâche afin d'organiser mon travail

US4 : En tant que professeur, je veux consulter les tâches des élèves afin de suivre leur progression

US5 : En tant qu'utilisateur, je veux un site rapide et simple afin de réduire la consommation de ressources

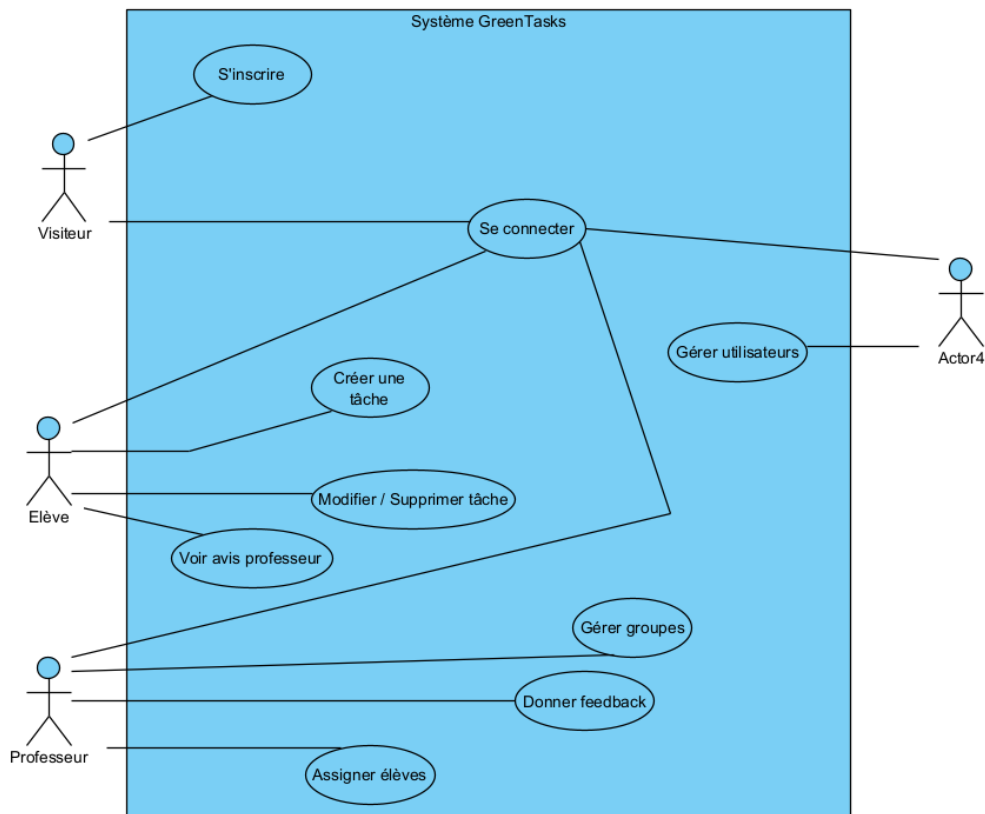
1.6 - Construire un mini backlog

Identifiant	Fonctionnalité	Priorité	Niveau de difficulté estimé	Personne / Sous-groupe
B1	Inscription (création de compte)	Haute	Facile	
B2	Connexion / Déconnexion	Haute	Facile	
B3	Création d'une tâche par un élève	Haute	Moyenne	
B4	Consultation des tâches par le professeur	Haute	Moyenne	
B5	Interface sobre, rapide et simple	Haute	Moyenne	

Phase 2 - Architecture fonctionnelle et conception

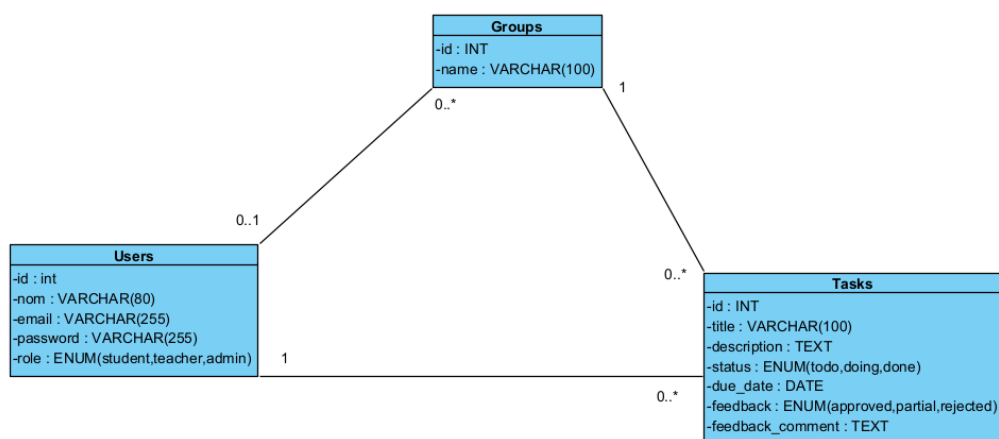
2.1 - Diagramme de cas d'utilisation

Il y a quatre utilisateurs qui ont cas d'utilisations différentes :



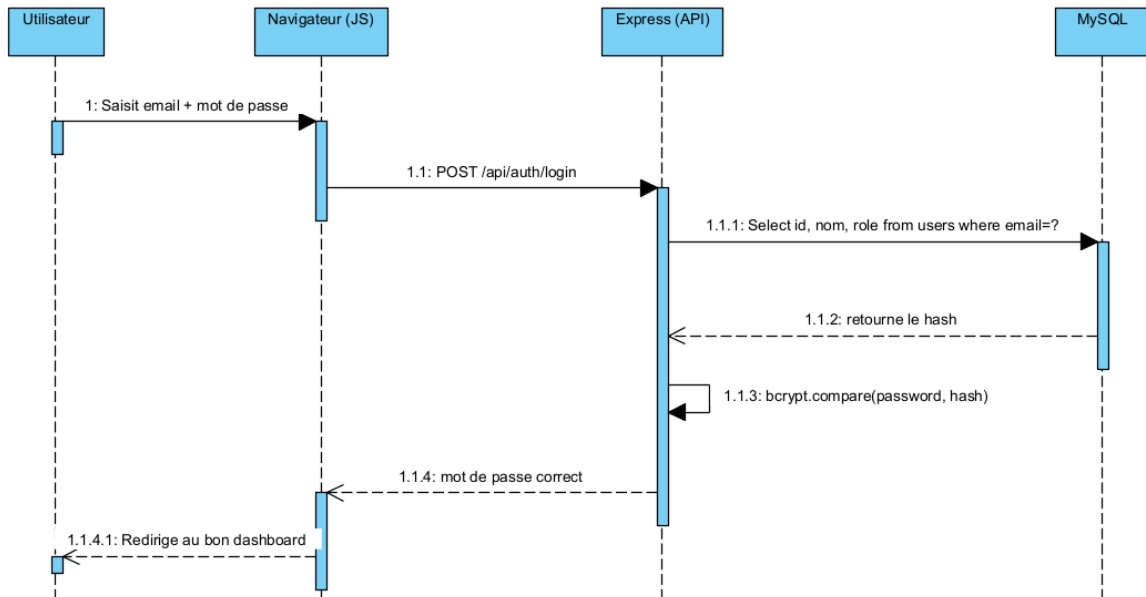
2.2 - Diagramme de classes

Il y a trois entités qui constituent le diagramme de classe :



2.3 - Diagramme de séquence - Connexion utilisateur

Ce scénario illustre les interactions lors de la connexion avec validation côté serveur, requête SQL, vérification bcrypt et redirection selon le rôle.



Phase 3 – Choix des technologies justifiées

Chaque technologie a été choisie selon cinq critères : simplicité, consommation de ressources, rapidité de mise en œuvre, facilité de maintenance et cohérence avec le périmètre du projet.

Composant	Choix	Simplicité	Ressources	Rapidité	Maintenance	Cohérence périmètre
Frontend	HTML/CSS/JS	Aucun outil à installer	0 Ko de framework	Ecriture directe	Fichiers lisible par tous	Seulement quelques pages statiques
Backend	Node.js/Express	Plus simple à configurer	Faible CPU	Démarrage rapide	Un seul fichier main	Idéal pour API REST
Base de données	MySQL	SQL standard	Requêtes rapides et ciblées	Fichier SQL prêt directement	Requêtes lisibles	Données structurées simples
Hashage	Bcryptjs	2 fonctions seulement : hash() + compare()	JS pas de compilation	Seulement 2 lignes de code	Bien documenté	Sécurité mots de passe suffisante
Déploiement	Github Pages	En quelque clique	Hébergeur reconnue	Déploiement auto sur push main	Logs intégrés	Gratuit

Phase 4 – Implémentation

4.1 - Développement front-end

Le front-end du site a été développé en HTML, CSS et JavaScript, sans aucun framework ou bibliothèque externe. Cela permet de charger des pages très légères, avec un minimum de requêtes réseau.

Pages créées

Page	Fichier	Contenu
Accueil	index.html	Présentation du projet et liens de connexion / inscription
Connexion	login.html	Formulaire email + mot de passe, message d'erreur si échec
Inscription	register.html	Formulaire avec choix du rôle et du groupe (pour les élèves)
Tableau de bord	dashboard.html	Interface adaptée selon le rôle : élève, enseignant ou admin

Optimisation des ressources statiques

HTML minimal, fichier < 15 Ko par page.

CSS minimal (style.css < 6 Ko).

JavaScript minimal, fichier < 10 Ko.

Aucune image dans l'interface - 0 Ko de médias décoratifs.

De plus nous avons utilisé le site minify CSS afin de minimaliser tous les fichiers CSS pour rendre les fichiers encore plus légères.

Chargement intelligent

Les données sont chargées via fetch() uniquement au moment où l'utilisateur en a besoin, ainsi il n'y a pas de chargement anticipé inutile.

Aucune police externe, seulement les polices système sont utilisées, ce qui évite toute requête vers Google Fonts.

Sobriété visuelle

Mode sombre automatique via la règle CSS @media (prefers-color-scheme: dark), ce qui réduit la consommation sur les écrans OLED

Aucune animation décorative, les seules transitions existantes ont une utilité fonctionnelle (affichage/masquage des sections selon le rôle).

Accessibilité de base

Utilisation de balises sémantiques : <main>, <header>, <footer>

La structure des titres (h1, h2) est logique et cohérente sur toutes les pages

4.2 - Développement back-end et accès à la base de données

Gestion des utilisateurs — CRUD complet obligatoire

La gestion des utilisateurs couvre les quatre opérations essentielles :

Opération	Détail
Création	Formulaire validé par le serveur avec l'email et mot de passe ≥ 6 caractères
Lecture	Les résultats sont paginés (20 par pages)
Modification	L'utilisateur peut modifier son propre nom
Suppression	Confirmation serveur pour suppression d'un utilisateur (seulement l'admin peut le faire)

Entité métier principale — CRUD complet

La tâche est l'entité métier principale du projet. Son CRUD est accessible uniquement aux élèves rattachés à un groupe. Les enseignants disposent en plus d'une fonctionnalité de feedback.

Opération	Détail
Création	Titre obligatoire, description, statut (à faire / en cours / terminé), échéance
Lecture	Liste paginée filtrée par statut pour l'élève
Modification	Possible uniquement par l'élève propriétaire de la tâche
Suppression	Possible uniquement par l'élève propriétaire de la tâche
Feedback	Enseignant (validée / partielle / refusée) avec un commentaire optionnel

Optimisation et sécurité des accès à la base de données

La base de données MySQL est initialisée via le fichier database/schema.sql qui contient toutes les instructions de création des tables, des contraintes et des index. La connexion est gérée via un pool de connexions (database/db.js), ce qui évite d'ouvrir une nouvelle connexion à chaque requête.

Nous avons utilisé des select avec des colonnes précises et non pas des select * afin de récupérer uniquement l'essentiel et donc de réduire la quantité de données transférées. La plupart des résultats ont été paginés (20 par pages). De plus, nous avons utilisé le hashage des mots de passe avec bcrypt pour mieux sécuriser les mots de passe. Quelques alertes ont également été mis en place afin de valider une quelconque action sensible.

4.3 - Déploiement du site web

Pour le déploiement du site web, on s'est orienté vers Render.com qui est très facilement accessible et gratuit pour déployer le site web. En complément à render.com, nous avons utilisés railway.app pour mettre notre base de données MySQL en ligne. Nous avons bien entendu utilisé .env listé dans .gitignore avec un fichier .env.example afin de documenter les variables attendues sans révéler les réelles valeurs.

URL pour le site : <https://greentasks-xl25.onrender.com/>

Phase 5 – Analyse de l'empreinte carbone

5.1 - Outils de mesure

Concernant les outils de mesure utilisés, on a retenu les suivants :

- Website Carbon Calculator : <https://www.websitecarbon.com>
- EcoIndex : <https://www.ecoindex.fr>
- PageSpeed Insights : <https://pagespeed.web.dev>
- WebPageTest : <https://www.webpagetest.org>
-

5.2 - Protocole d'analyse

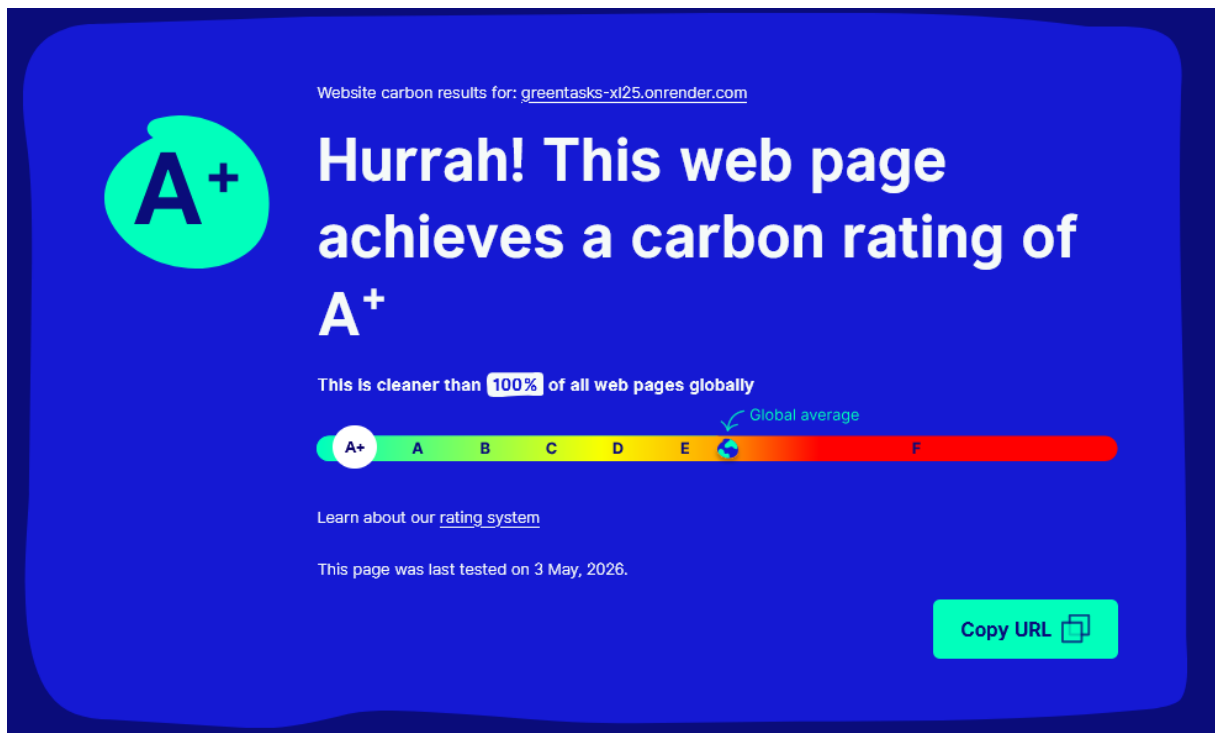
Avant le développement du site web, on a analysé les différentes sources potentielles de consommation lourdes. Cette analyse nous a guidé dans nos choix d'implémentation dès le départ.

Source identifiée	Impact	Décision prise
Frameworks CSS (Bootstrap ~30 Ko)	Poids inutile	CSS natif < 6 Ko
Google Fonts (1 req. par police)	Requête externe + latence	Polices système uniquement
SELECT * systématique	Surcharge BDD et réseau	SELECT colonnes exactes
Scroll infini (chargement continu)	Requêtes BDD non contrôlées	Pagination LIMIT 20
Animations CSS/JS décoratives	Affichage paginé correct	Aucune animation
Images décoratives	Poids médias	Aucune image dans l'UI
Dépendances npm superflues	Bundle lourd en prod	5 paquets max

D'après les mesures prises avec les différents sites données, nous obtenons ces résultats :

Website Carbon Calculator :

<https://www.websitecarbon.com/website/greentasks-xl25-onrender-com/>



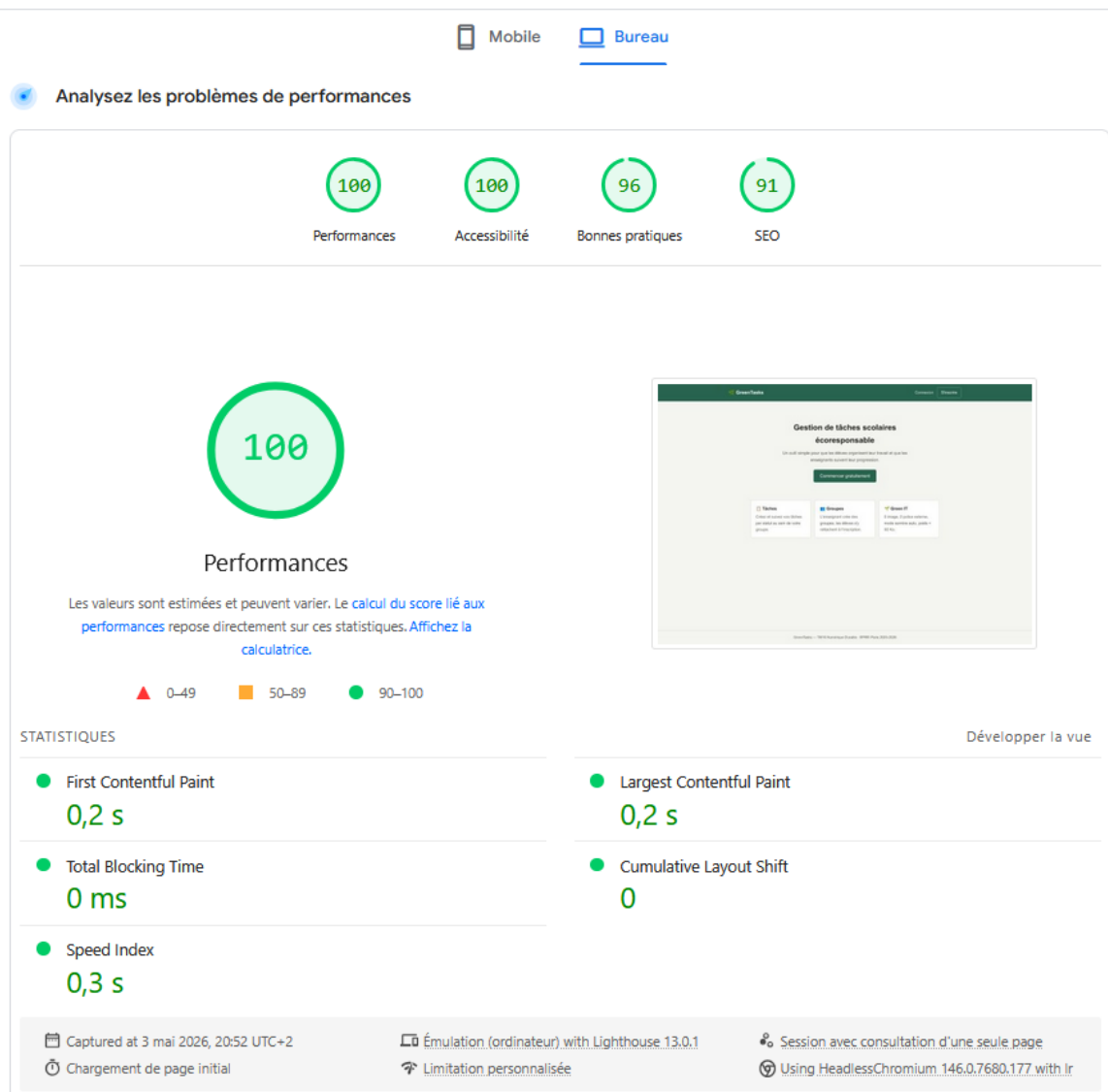
EcoIndex :

<https://www.ecoindex.fr/resultat/?id=8eac1c33-1f5a-4ed3-84d6-b674557323a0>



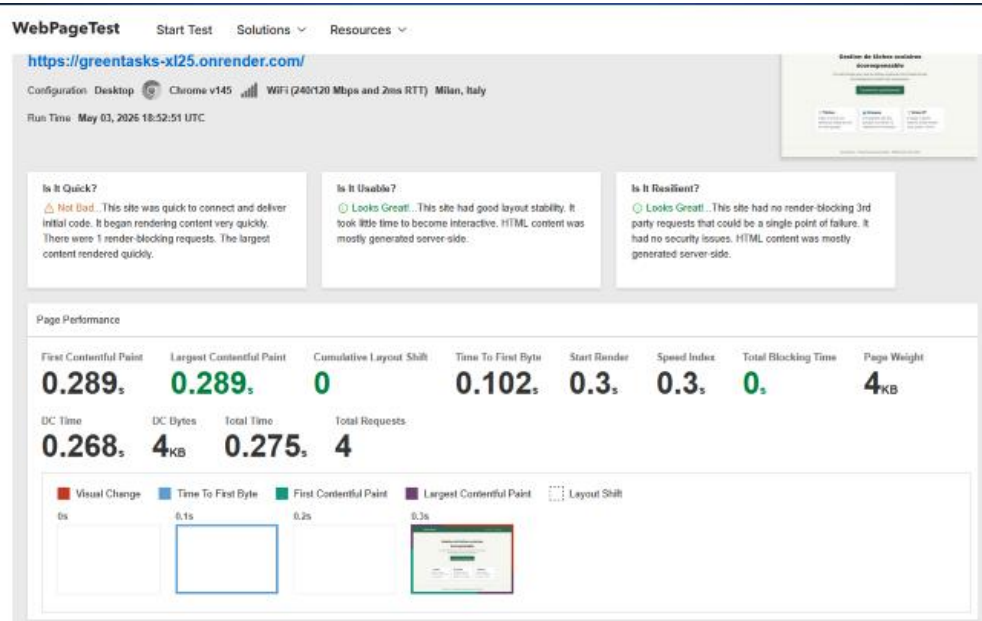
PageSpeed Insights :

https://pagespeed.web.dev/analysis/https-greentasks-xl25-onrender-com/y7qnic9eid?form_factor=desktop



WebPageTest :

<https://www.catchpoint.com/webpagetest/results?publicurl=https%3A%2F%2Fpublic.catchpoint.com%2FUI%2Fentry%2FWPTITP%2FARRs-D-E-B2ACNAcjqKbrXYAA-N>



D'après les différents résultats obtenus, on peut donc dresser ce tableau :

Indicateur	Valeur
Poids de la page d'accueil	4 Ko
Nombre de requêtes HTTP	3
Score EcoIndex	96/100
Note EcoIndex	A
CO2 / visite	0,01g / visite
Score Lighthouse Perf.	97,5/100 (100 + 100 + 96 + 91)
FCP	0,3 s
LCP	0,3 s

Ainsi, comparé aux sites concurrents comme Monday.com ou bien Notion.so, on remarque qu'à travers des tests comme EcoIndex, les scores de ces deux sont autour de 5/100 avec une note de G. Contrairement à GreenTasks qui lui est à 96/100 avec une note A.

Résultats de l'analyse

<https://monday.com/>



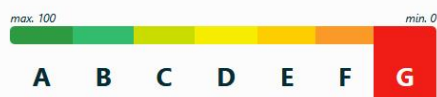
Outch.

Score : 5 / 100



[Que veut dire mon score](#) ▼

On ne va pas se le cacher : ça fait mal. Il est temps d'agir !



Classement de la page : 591039 / 594050

- Trop lourde 19,589 Mo
- Trop complexe 4008 éléments
- Trop de requêtes 377 requêtes

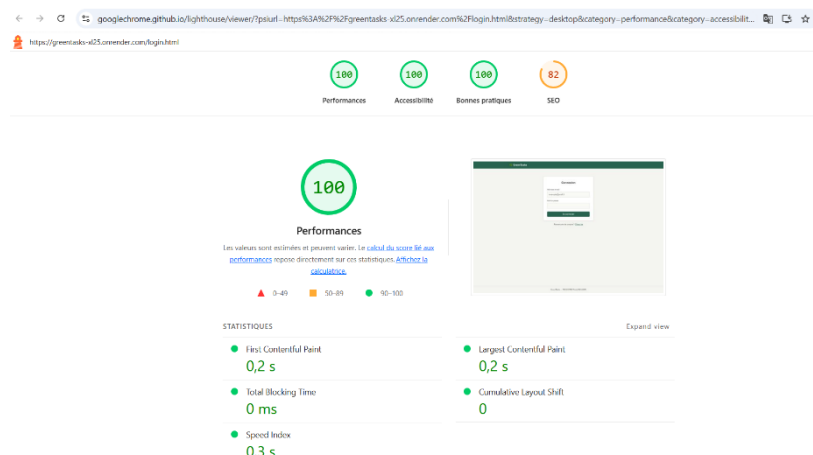
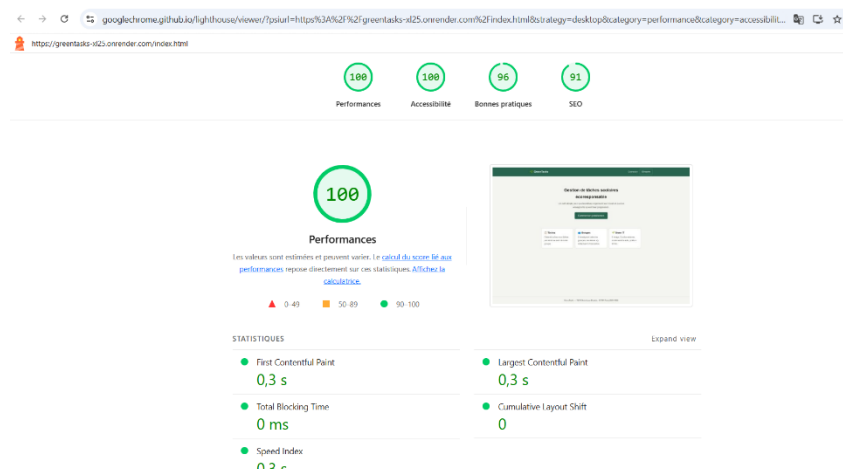
[Voir les détails du score](#) >

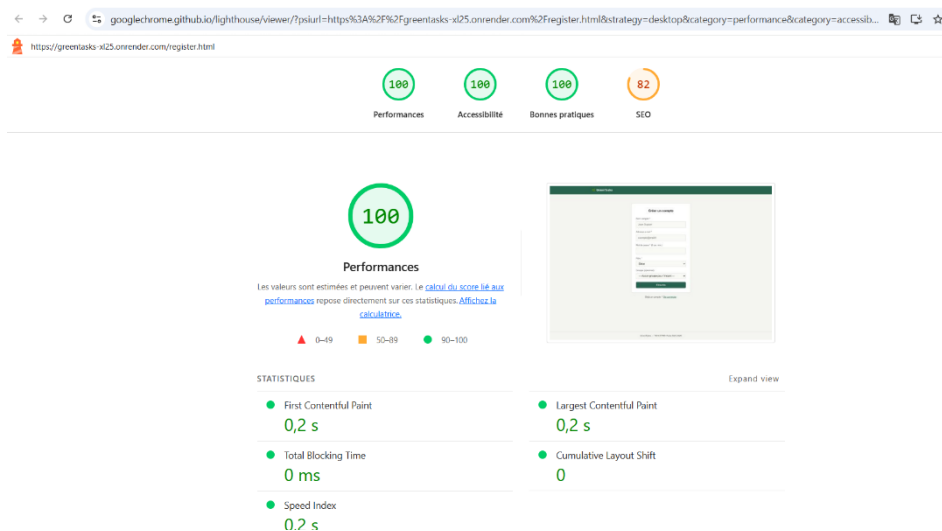
Phase 6 — Tests et validation

6.1 - Tests fonctionnels

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Utilisateur enregistré en BDD, redirection	Ok
Créer un utilisateur (email vide)	Message d'erreur côté serveur	Ok
Modifier un utilisateur existant	Données mises à jour en BDD	Ok
Supprimer un utilisateur	Utilisateur supprimé après confirmation	Ok
Lister les utilisateurs	Affichage paginé correct	Ok
Créer une entité métier	Entité enregistrée, liste mise à jour	Ok
Connexion avec identifiants valides	Redirection vers espace personnel	Ok
Connexion avec mauvais MDP	Message d'erreur, pas de session créée	Ok
Accès page protégée sans connexion	Redirection vers la page de connexion	Ok

6.2 - Tests de performance





De plus, même en testant le site sur un réseau simulé 3G lent, le site tout a fait fonctionnel avec une vitesse normale.

6.3 - Tests de sécurité minimaux

Les mots de passe sont bien sécurisé et ne sont pas en clair dans la base de données.

```
1 • SELECT password FROM users;
```

password
\$2a\$10\$oiFko4ZQr3TZyZbY19kOIu0Rv4Ty8Swt...
\$2a\$10\$gtKLgMQHOE8fyTYaM2SgPeRTBaZpUcr...
\$2a\$10\$jpwqe7y6rHs5/9YQfgOjIOId/g.UmgYob...
\$2a\$10\$ud1gePZh8g8.djflTSu0aefyFMN6N8xW...
\$2a\$10\$kc.4fZZ4Ii.qleRPrZ8xHeRKxJZ6PVce7T...

De plus, il n'y a aucune variable sensible dans Git et les formulaires rejettent les entrées malformées :

Adresse e-mail *

'''

! Veuillez inclure "@" dans l'adresse e-mail. Il manque un symbole "@" dans "'''".

Et enfin, un utilisateur non connecté ne peut pas accéder aux pages protégées en modifiant l'URL directement. En effet, par exemple nous sommes dans l'index de base (index.html), on ne peut pas le remplacer et mettre dashbord.html pour être dans la vue d'un élève, d'un enseignant ou même d'un admin. Il sera directement redirigé au login.html.